

LAB WORKBOOK

Assignment 5: The HTML Web Table Extractor

Tier 2: Advanced User Interactions

1. Module Name and Experiment Title

Module: Tier 2 – Advanced User Interactions

Experiment: The HTML Web Table Extractor

2. Problem Statement

Automate a web application containing a multi-column data table. Iterate through the table rows and columns, locate a row by matching a product name, and retrieve the corresponding value from the Price column. The solution must work by reading the table structure rather than relying on a hard-coded row number.

3. Objective

To learn how to extract structured data from HTML tables using Selenium by iterating through rows and columns, identifying a target row, and retrieving a related column value.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Website	SauceDemo – https://www.saucedemo.com/
Concepts	HTML tables, rows, columns, XPath, loops, text matching, data extraction, assertions

5. Source Code / Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    # Open SauceDemo
    driver.get("https://www.saucedemo.com/")

    # Login
    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.NAME, "password").send_keys("secret_sauce")
```

```

driver.find_element(By.XPATH, "//input[@id='login-button']").click()

# Wait until the inventory page is displayed
products = driver.find_elements(By.CSS_SELECTOR, ".inventory_item")

target_name = "Sauce Labs Backpack"
target_price = None

# Iterate through product rows
for product in products:
    name = product.find_element(
        By.CSS_SELECTOR, ".inventory_item_name"
    ).text
    price = product.find_element(
        By.CSS_SELECTOR, ".inventory_item_price"
    ).text

    print("Product:", name, "| Price:", price)

    # Match the required row
    if name == target_name:
        target_price = price
        break

# Validate and display the extracted value
assert target_price is not None, "Target product was not found."

print("Matching product:", target_name)
print("Price:", target_price)
print("Validation passed: target row located successfully.")

finally:
    driver.quit()

```

6. Implementation Steps

1. Open the SauceDemo login page and sign in.
2. Locate all inventory item rows as the table-like data records.
3. Loop through each record one by one.
4. Read the product name from the name column and the value from the Price column.
5. Compare the product name with the target name 'Sauce Labs Backpack'.
6. When the matching row is found, store its corresponding Price value.
7. Print the matched product and extracted price, validate that a value was found, and close the browser.

7. Output

```
[Running] python -u "c:\Users\SUTANU\Desktop\selenium\ass5.py"  
Product: Sauce Labs Backpack | Price: $29.99  
Matching product: Sauce Labs Backpack  
Price: $29.99  
Validation passed: target row located successfully.
```

8. Result

The program successfully iterated through the inventory records, located the row matching the selected product name, and extracted its corresponding Price value.

9. Observation

Iterating through records and matching text is more reliable than depending on a fixed row position. Once the target record is found, values from related columns can be extracted directly.

10. Conclusion

This experiment demonstrated Selenium-based extraction of structured web data by iterating through records, matching a specific row, and retrieving a corresponding column value.